National University of Sciences & Technology
Military College of Signals
Department of Information Security
Machine Learning

Heart Disease Prediction Using a Custom Artificial Neural Network

Project Report

S.No	Group Members
1.	NC Ali Irshad Ch
2.	NC Haider Nawaz
3.	NC Haris Abdullah
4.	NC Huzaifa Asim

Introduction

This project presents a binary classification system designed to predict the presence of heart disease using clinical and medical attributes from a heart disease dataset. The main objective is to classify whether a patient is likely to have heart disease based on input parameters such as age, chest pain type, resting blood pressure, cholesterol level, maximum heart rate, exercise-induced angina, and other medical indicators.

The artificial neural network was implemented from scratch using NumPy instead of deep learning frameworks such as TensorFlow or PyTorch. This allows each major part of the learning process to be clearly demonstrated, including model initialization, forward propagation, sigmoid activation, binary cross-entropy loss calculation, backpropagation, and gradient descent-based weight updates.

The project follows the required neural network design: four hidden layers, each containing seven neurons, with sigmoid activation applied across the network. The final model is evaluated using standard classification metrics including accuracy, precision, recall, F1 score, and a confusion matrix.

The key goals of this project are:

1. Design and implement a multi-layer artificial neural network from scratch.
2. Use the heart disease dataset for binary classification.
3. Apply preprocessing and feature scaling before training.
4. Demonstrate forward propagation, loss computation, backpropagation, and optimization.
5. Evaluate the trained model using precision, recall, F1 score, and confusion matrix analysis.

Dataset

Overview

The dataset used in this project is a heart disease dataset stored in CSV format as heart.csv. It contains 14 columns in total: 13 input features and 1 target column. Each row represents a patient record, and the target column indicates whether heart disease is present or not.

Dataset Features

Feature	Type	Description
age	Numerical	Age of the patient
sex	Binary/Categorical	Sex of the patient
cp	Categorical	Chest pain type
trestbps	Numerical	Resting blood pressure
chol	Numerical	Serum cholesterol level
fbs	Binary	Fasting blood sugar greater than 120 mg/dl
restecg	Categorical	Resting electrocardiographic results
thalach	Numerical	Maximum heart rate achieved
exang	Binary	Exercise-induced angina
oldpeak	Numerical	ST depression induced by exercise relative to rest

slope	Categorical	Slope of the peak exercise ST segment
ca	Numerical/Categorical	Number of major vessels colored by fluoroscopy
thal	Categorical	Thalassemia-related result
target	Binary Target	0 = No heart disease, 1 = Heart disease

Preprocessing

6. The CSV file was loaded using pandas with pd.read_csv("heart.csv").
7. The 13 medical parameters were selected as input features, while target was selected as the output label.
8. Missing values were checked and handled to prevent errors during training.
9. The dataset was not filtered by removing duplicate records, so the records remain consistent with the provided CSV file.
10. The target column was reshaped into a two-dimensional vector to match the neural network output shape.
11. Feature scaling was performed using StandardScaler so that numerical features have a balanced scale during gradient descent.

Train/Test Split

The dataset was divided into training and testing sets using an 80/20 split. Stratified splitting was applied so that both training and testing sets preserve the class distribution of the target column. This helps make the evaluation more reliable for binary classification.

Data Split	Percentage	Purpose
Training Set	80%	Used to train the ANN and update model weights
Testing Set	20%	Used to evaluate the final model on unseen data

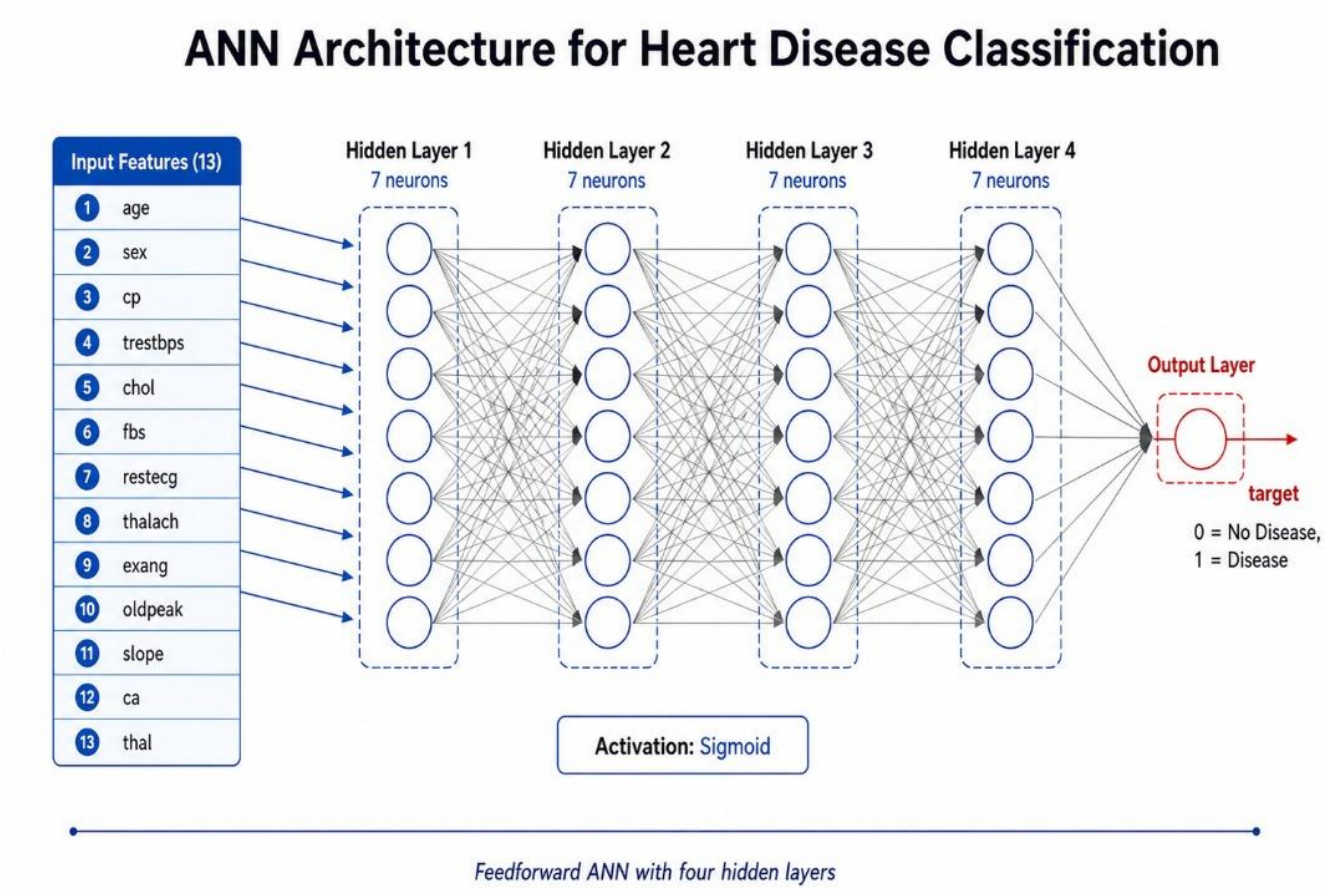
Neural Network Architecture

Network Structure

The neural network is a fully connected feedforward artificial neural network. It contains one input layer, four hidden layers, and one output layer. Each hidden layer contains exactly seven neurons, fulfilling the project requirement.

Layer	Type	Neurons	Activation
Layer 0	Input	13	—
Layer 1	Hidden	7	Sigmoid

Layer 2	Hidden	7	Sigmoid
Layer 3	Hidden	7	Sigmoid
Layer 4	Hidden	7	Sigmoid
Layer 5	Output	1	Sigmoid



Weight Matrices and Bias Vectors

Matrix	Shape	Bias Shape	Connection
W1, b1	(13, 7)	(1, 7)	Input → Hidden Layer 1
W2, b2	(7, 7)	(1, 7)	Hidden Layer 1 → Hidden Layer 2
W3, b3	(7, 7)	(1, 7)	Hidden Layer 2 → Hidden Layer 3
W4, b4	(7, 7)	(1, 7)	Hidden Layer 3 → Hidden Layer 4
W5, b5	(7, 1)	(1, 1)	Hidden Layer 4 → Output

			Layer

Weights were initialized using random normal values scaled according to the input size of each layer, and all bias vectors were initialized to zero. A fixed random seed was used to make the experiment reproducible.

Methods

Activation Function: Sigmoid

The sigmoid function was used as the activation function for all hidden layers and the output layer. It maps any real-valued input to a value between 0 and 1, making it suitable for binary classification tasks.

Sigmoid function: $\sigma(z) = 1 / (1 + e^{(-z)})$

Sigmoid derivative: $\sigma'(a) = a \times (1 - a)$

To prevent numerical overflow during exponential calculation, the input z was clipped to a safe range before applying the sigmoid function.

Forward Propagation

Forward propagation is the process of passing input data through the network to produce predictions. For each layer, the model first calculates a weighted sum and then applies the sigmoid activation function.

For each layer: $Z = A_{prev} \cdot W + b$

Activation: $A = \sigma(Z)$

This process is repeated from the first hidden layer to the output layer. The final output A5 represents the predicted probability of heart disease.

Loss Function: Binary Cross-Entropy

Binary cross-entropy was used to measure the difference between predicted probabilities and actual target labels. This loss function is appropriate because the problem is binary classification.

Loss: $L = -(1/m) \times \Sigma [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$

A small epsilon value was added inside logarithms to avoid mathematical errors caused by log(0).

Backpropagation

Backpropagation calculates how much each weight and bias contributed to the prediction error. The gradients are computed using the chain rule, starting from the output layer and moving backward through the hidden layers.

For the output layer: $dZ5 = A5 - y$

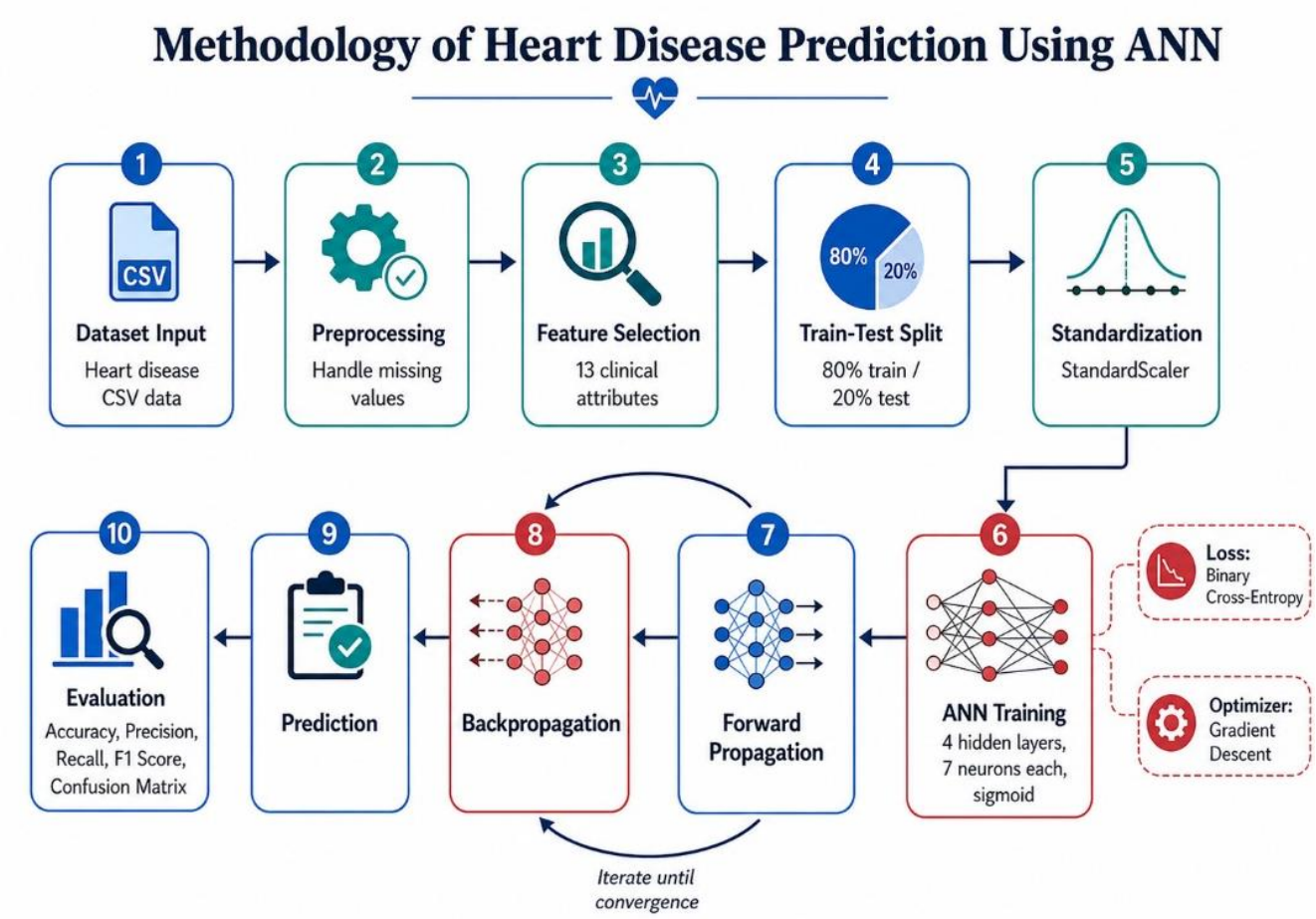
For each layer: $dW = (A_{prev}^T \cdot dZ) / m, \quad db = \Sigma(dZ) / m$

For hidden layers, the error is multiplied by the derivative of the sigmoid activation function to determine how much each hidden neuron contributed to the final error.

Gradient Descent

After calculating gradients, weights and biases are updated using gradient descent. The learning rate controls how large each update step is.

Parameter update: $W \leftarrow W - \alpha \times dW, \quad b \leftarrow b - \alpha \times db$



Training

The training loop repeatedly performs forward propagation, loss calculation, backpropagation, and parameter updates. The model was trained for 5000 epochs using full-batch gradient descent.

Hyperparameter	Value
Epochs	5000
Learning Rate	0.1
Hidden Layers	4

Neurons per Hidden Layer	7
Activation Function	Sigmoid
Loss Function	Binary Cross-Entropy
Optimizer	Gradient Descent with Backpropagation
Batch Type	Full batch
Random Seed	42

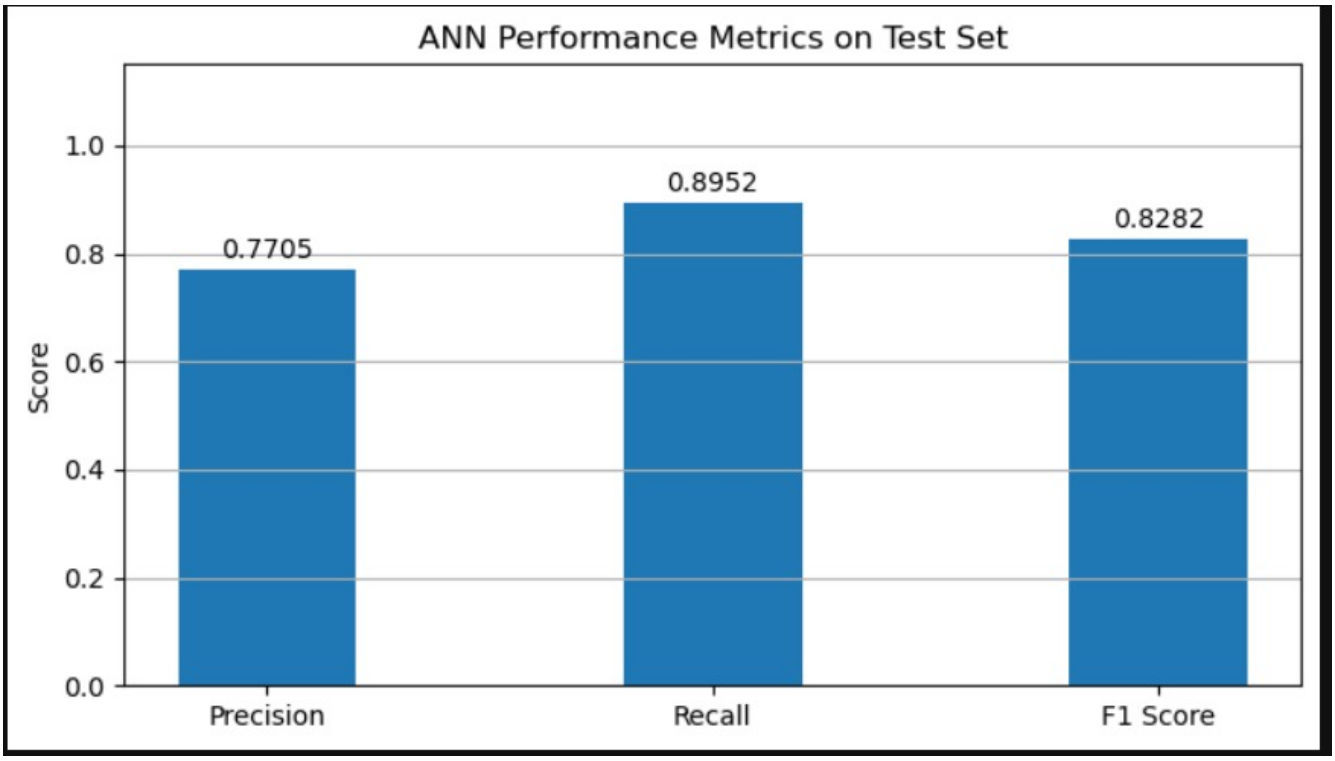
During training, the loss value was stored at every epoch and displayed periodically. A loss curve was generated to show whether the neural network was learning over time.

Results and Evaluation

Performance Metrics

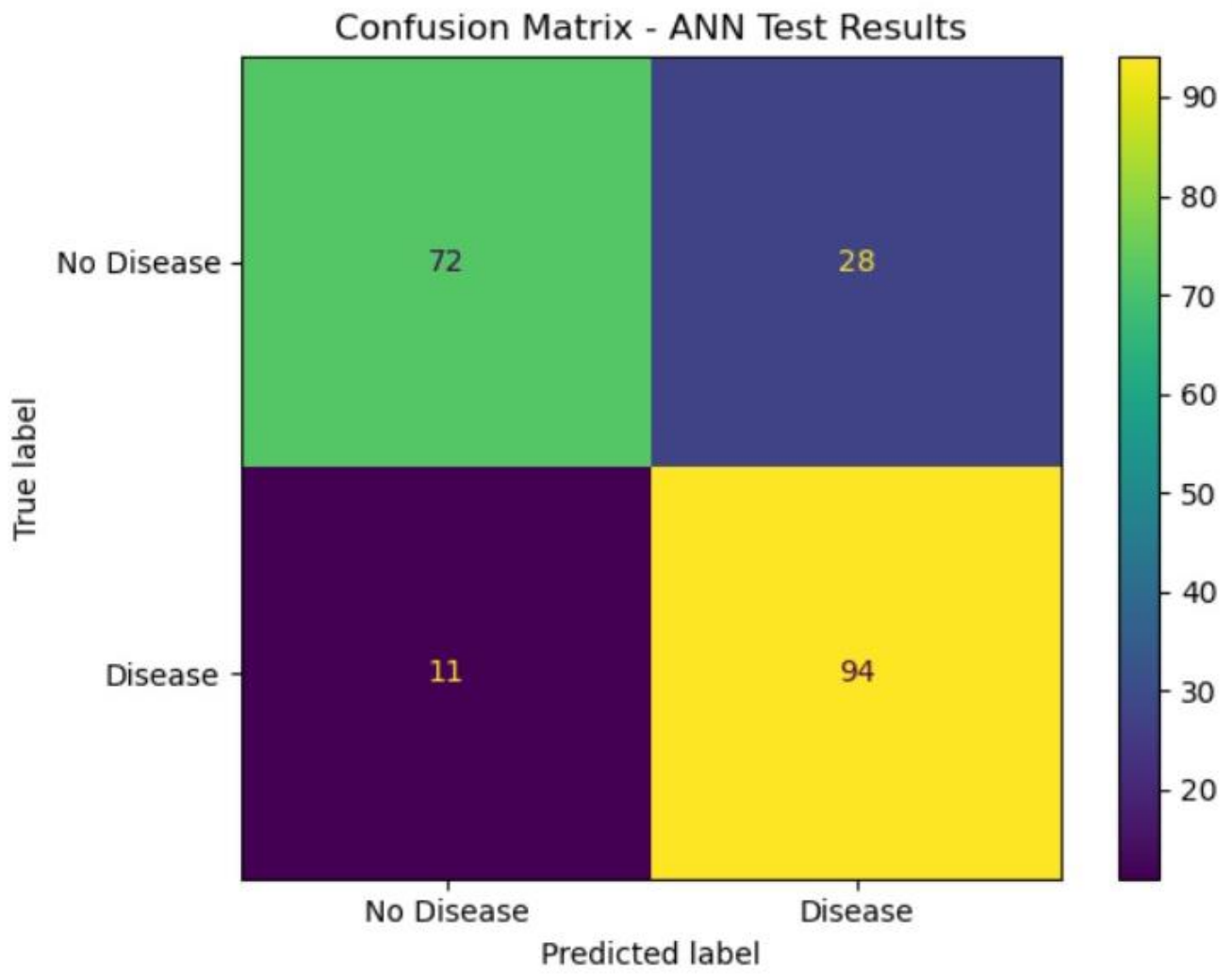
The trained ANN was evaluated on the test set using accuracy, precision, recall, and F1 score. These metrics provide a more complete picture of classification performance than accuracy alone.

Metric	Value from Final Notebook Output
Training Accuracy	82.44%
Testing Accuracy	80.98%
Precision	0.7705
Recall	0.8592
F1 Score	0.8282



Confusion Matrix

The confusion matrix summarizes the number of correct and incorrect predictions made by the model. It separates predictions into true positives, true negatives, false positives, and false negatives.




```
Confusion Matrix Values:
True Negatives  : 72
False Positives : 28
False Negatives : 11
True Positives  : 94
```

Metric Interpretation

Precision shows how many of the patients predicted as having heart disease were actually positive cases. Recall shows how many actual heart disease cases the model successfully detected. F1 score balances precision and recall, making it useful when both false positives and false negatives matter.

In a medical prediction problem, recall is especially important because missing a patient with heart disease can be more serious than incorrectly flagging a healthy patient. However, precision is also important because too many false alarms can reduce trust in the system and increase unnecessary follow-up testing.

Analysis and Discussion

The ANN successfully follows a complete machine learning workflow: loading the dataset, preprocessing input features, initializing weights and biases, performing forward propagation, calculating loss, applying backpropagation, updating parameters, and evaluating the model on unseen test data.

The use of four hidden layers with sigmoid activation satisfies the project requirements, but it can also create learning challenges. Sigmoid functions can suffer from the vanishing gradient problem when used in several stacked layers. This means earlier layers may receive very small gradient values, causing slow learning or limited improvement during training.

Feature scaling is important in this project because the dataset includes variables with different numeric ranges, such as cholesterol, blood pressure, and maximum heart rate. Standardization helps the gradient descent process move more smoothly and prevents larger-scale features from dominating the training process.

Challenges and Limitations

12. The model uses sigmoid activation in all hidden layers, which can lead to vanishing gradients in deeper networks.
13. The model is implemented using full-batch gradient descent, which may train more slowly than mini-batch gradient descent.
14. The dataset size and quality directly affect performance, especially in medical prediction tasks.
15. The network does not use advanced regularization techniques such as dropout or L2 regularization.
16. The ANN provides predictions but does not replace professional medical diagnosis.

Conclusion

This project demonstrates the design and implementation of an artificial neural network for heart disease prediction using NumPy. The model was built from scratch and includes all major stages of ANN training, including forward propagation, sigmoid activation, binary cross-entropy loss, backpropagation, and gradient descent optimization.

The final architecture uses 13 input features, four hidden layers with seven neurons each, and one sigmoid output neuron for binary classification. The trained model is evaluated using accuracy, precision, recall, F1 score, and confusion matrix analysis, providing a structured understanding of its predictive performance.

17. Future improvements could include experimenting with different learning rates, using mini-batch gradient descent, applying regularization, comparing sigmoid with ReLU activation, and testing the model using

cross-validation for more reliable performance estimation.

```
============================================================
ANN ARCHITECTURE SUMMARY
============================================================
Input Layer Features      : 13
Hidden Layers             : 4
Neurons per Hidden Layer  : 7
Output Layer Neurons      : 1
Activation Function       : Sigmoid
Loss Function             : Binary Cross-Entropy
Optimizer                 : Gradient Descent with Backpropagation
Learning Rate             : 0.1
Epochs                    : 5000
Total Parameters          : 274
============================================================
```

Tools and Libraries

Tool / Library	Purpose
NumPy	Matrix operations, weight initialization, forward propagation, backpropagation, and gradient descent
Pandas	Reading and handling the CSV dataset
scikit-learn	Train/test split, feature scaling, and evaluation metrics
Matplotlib	Plotting the training loss curve and metrics chart
Jupyter Notebook	Development and execution environment